main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
struct Node* head = NULL;

// Insert at beginning
void insertStart(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL)
        head->prev = newNode;

    head = newNode;
}
// Insert at end
void insertEnd(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* temp = head;

    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
```

Output

Clear

```
List: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    temp->next = newNode;
    newNode->prev = temp;
}
// Insert at position
void insertPosition(int data, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* temp = head;

    newNode->data = data;

    if (pos == 1) {
        insertStart(data);
        return;
    }
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Position not found\n");
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}
// Display list
void display() {
    struct Node* temp = head;

    printf("List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
```

Output

Clear

```
List: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

Programiz
C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
    if (temp == NULL) {
        printf("Position not found\n");
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}
// Display list
void display() {
    struct Node* temp = head;

    printf("List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
// Main function
int main() {

    insertStart(10);
    insertStart(5);

    insertEnd(20);
    insertEnd(30);

    insertPosition(15, 3);

    display();

    return 0;
}
```

Output

Clear

```
List: 5 <-> 10 <-> 15 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

// Delete at beginning
void delete_begin()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;
    head = head->next;

    if(head != NULL)
        head->prev = NULL;

    free(temp);
}

// Delete at end
void delete_end()
{
    struct node *temp;

    if(head == NULL)
```

Output

Clear

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    if(head == NULL)
    {
        printf("List empty\n");
        return;
    }

    temp = head;
    while(temp->next != NULL)
        temp = temp->next;

    if(temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL;

    free(temp);
}

// Delete at position
void delete_middle(int pos)
{
    struct node *temp;
    int i;

    temp = head;

    for(i=1;i<pos;i++)
        temp = temp->next;

    temp->prev->next = temp->next;

    if(temp->next != NULL)
        temp->next->prev = temp->prev;

    free(temp);
}

// Display list
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
// Display list
void display()
{
    struct node *temp = head;

    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main()
{
    struct node *n1,*n2,*n3,*n4;
    // Creating nodes
    n1 = (struct node*)malloc(sizeof(struct node));
    n2 = (struct node*)malloc(sizeof(struct node));
    n3 = (struct node*)malloc(sizeof(struct node));
    n4 = (struct node*)malloc(sizeof(struct node));

    // Assign data
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    n4->data = 40;

    // Linking nodes
    n1->prev = NULL;
    n1->next = n2;

    n2->prev = n1;
    n2->next = n3;

    n3->prev = n2;
    n3->next = n4;
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```

main.c

Share Run

```c
    // Assign data
    n1->data = 10;
    n2->data = 20;
    n3->data = 30;
    n4->data = 40;

    // Linking nodes
    n1->prev = NULL;
    n1->next = n2;

    n2->prev = n1;
    n2->next = n3;

    n3->prev = n2;
    n3->next = n4;

    n4->prev = n3;
    n4->next = NULL;

    head = n1;

    printf("Original List:\n");
    display();

    delete_begin();
    printf("After delete begin:\n");
    display();

    delete_middle(2);
    printf("After delete middle:\n");
    display();

    delete_end();
    printf("After delete end:\n");
    display();

    return 0;
}
```

Output

```
Original List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
After delete begin:
20 <-> 30 <-> 40 <-> NULL
After delete middle:
20 <-> 40 <-> NULL
After delete end:
20 <-> NULL


=== Code Execution Successful ===
```